Cairo 실행 환경과 VM 분석

개요

Cairo 실행 환경은 컴파일된 CASM 코드를 실제로 실행하고 증명을 생성하는 핵심 컴포넌트입니다. `cairo-lang-runner` crate는 Sierra 프로그램을 Cairo VM에서 실행하기 위한 기본 러너를 제공하며, 메모리 관리, 가스 계산, 힌트 처리, 그리고 Starknet 특화 기능들을 포함합니다.

Cairo VM 인터페이스와 실행 모델

1. SierraCasmRunner 구조

`SierraCasmRunner` 는 Sierra 프로그램을 CASM으로 변환하고 실행하는 메인 인터페이스입니다:

```
pub struct SierraCasmRunner {
/// 실행 가능한 함수를 위한 빌더
builder: RunnableBuilder,
/// class_hash에서 contract info로의 매핑
starknet_contracts_info: OrderedHashMap<Felt252, ContractInfo>,
/// 프로파일러 실행 여부
run_profiler: Option<ProfilingInfoCollectionConfig>,
}
```

2. 실행 흐름

2.1 함수 실행 준비

```
pub fn prepare_starknet_context(
&self,
func: &Function,
args: Vec<Arg>,
available_gas: Option<usize>,
starknet_state: StarknetState,
) -> Result<(CairoHintProcessor<'_>, PreparedStarknetContext), RunnerError>
```

- Sierra 함수를 CASM 프로그램으로 어셈블
- 힌트 딕셔너리 구성
- 사용자 인수 준비
- 힌트 프로세서 초기화

2.2 VM 실행

```
pub fn run_function<'a, Bytecode>(
&self,
func: &Function,
hint_processor: &mut dyn HintProcessor,
hints_dict: HashMap<usize, Vec<HintParams>>,
bytecode: Bytecode,
builtins: Vec<BuiltinName>,
```

```
) -> Result<RunResult, RunnerError>
```

3. VM 초기화

```
pub fn initialize_vm(vm: &mut VirtualMachine, data_len: usize) -> Result<(), B
// 빌트인 비용 세그먼트 생성
let builtin_cost_segment = vm.add_memory_segment();
// 각 토큰 타입에 대한 가스 비용 설정
for token_type in CostTokenType::iter_precost() {
vm.insert_value(
(builtin_cost_segment + (token_type.offset_in_builtin_costs() as usize)).unwrap(
Felt252::from(token_gas_cost(*token_type)),
)?;
}
// 프로그램 끝에 빌트인 비용 세그먼트 포인터 배치
vm.insert_value((vm.get_pc() + data_len).unwrap(), builtin_cost_segment)?;
Ok(())
}
```

메모리 관리 메커니즘

1. 메모리 세그먼트 구조

Cairo VM은 세그먼트 기반 메모리 모델을 사용합니다:

- **프로그램 세그먼트**: CASM 바이트코드 저장
- **실행 세그먼트**: 스택과 힙 데이터
- **빌트인 세그먼트**: 각 빌트인별 전용 메모리 영역
- **임시 세그먼트**: 힌트 실행 중 생성되는 데이터

2. MemBuffer 헬퍼

연속적인 메모리 읽기/쓰기를 위한 헬퍼 구조체:

```
pub struct MemBuffer<'a> {
vm: &'a mut dyn VMWrapper,
pub ptr: Relocatable,
}

impl<'a> MemBuffer<'a> {
// 새 세그먼트 생성 및 버퍼 래핑
pub fn new_segment(vm: &'a mut dyn VMWrapper) -> Self
// 다음 위치의 felt252 값 읽기
pub fn next_felt252(&mut self) -> Result<Cow<'_, Felt252>, MemoryError>
// 값을 현재 위치에 쓰기
pub fn write<T: Into<MaybeRelocatable>>(&mut self, value: T) -> Result<(), Me
// 배열을 새 세그먼트에 쓰고 시작/끝 포인터 반환
pub fn write_arr<T, Data>(&mut self, data: Data) -> Result<(), MemoryError>
}
```

3. 메모리 접근 패턴

```
// 셀 참조를 통한 메모리 접근
fn get_cell_val(vm: &VirtualMachine, cell: &CellRef) -> Result<Felt252, Virtual


// 포인터를 통한 간접 접근
fn get_double_deref_val(
vm: &VirtualMachine,
cell: &CellRef,
offset: &Felt252,
) -> Result<Felt252, VirtualMachineError>


// 버퍼 추출 및 재배치 가능한 주소 변환
pub fn extract_relocatable(
vm: &VirtualMachine,
buffer: &ResOperand,
) -> Result<Relocatable, VirtualMachineError>
```

가스 계산 메커니즘

1. 토큰 기반 가스 모델

Cairo는 다양한 연산 타입에 대해 토큰 기반 가스 시스템을 사용합니다:

```
pub fn token_gas_cost(token_type: CostTokenType) -> usize {
match token_type {
CostTokenType::Const => 1,
CostTokenType::Pedersen => 4050,
CostTokenType::Poseidon => 491,
CostTokenType::Bitwise => 583,
CostTokenType::EcOp => 4085,
CostTokenType::AddMod => 230,
CostTokenType::MulMod => 604,
// Step, Hole, RangeCheck 등은 직접적인 가스 비용 없음
}
}
```

2. 함수별 가스 계산

```
pub fn initial_required_gas(&self, func: &Function) -> Option<usize> {
let gas_info = &self.builder.metadata().gas_info;
require(!gas_info.function_costs.is_empty())?;
Some(
gas_info.function_costs[&func.id]
.iter()
.map(|(token_type, val)| {
val.into_or_panic::<usize>() * token_gas_cost(*token_type)
```

```
})
.sum(),
)
}
```

3. Syscall 가스 비용

Starknet syscall들은 각각 고정된 가스 비용을 가집니다:

```
mod gas_costs {
pub const CALL_CONTRACT: usize = 10 * STEP + ENTRY_POINT;
pub const DEPLOY: usize = 200 * STEP + ENTRY_POINT;
pub const EMIT_EVENT: usize = 10 * STEP;
pub const GET_BLOCK_HASH: usize = 50 * STEP;
pub const STORAGE_READ: usize = 50 * STEP;
pub const STORAGE_WRITE: usize = 50 * STEP;
pub const SECP256K1_MUL: usize = 121810 * STEP + 10739 * RANGE_CHECK;
// ... 기타 syscall 비용들
}
```

4. 가스 차감 매크로

```
macro_rules! deduct_gas {
($gas:ident, $amount:ident) => {
```

```
if *$gas < gas_costs::$amount {
fail_syscall!(b"Syscall out of gas");
}
*$gas -= gas_costs::$amount;
};
}
```

힌트 처리 시스템

1. CairoHintProcessor

힌트 실행을 담당하는 메인 프로세서:

```
pub struct CairoHintProcessor<'a> {
pub runner: Option<&'a SierraCasmRunner>,
pub user_args: Vec<Vec<Arg>>,
pub string_to_hint: HashMap<String, Hint>,
pub starknet_state: StarknetState,
pub run_resources: RunResources,
pub syscalls_used_resources: StarknetExecutionResources,
pub no_temporary_segments: bool,
pub markers: Vec<Vec<Felt252>>,
pub panic_traceback: Vec<(Relocatable, Relocatable)>,
```

```
}
```

2. 힌트 타입별 처리

```
impl HintProcessorLogic for CairoHintProcessor<'_> {
fn  execute_hint(&mut  self, vm: &mut VirtualMachine, exec_scopes: &mut Executio
hint_data: &Box<dyn Any>, _constants: &HashMap<String, Felt252>) -> Result<(), H
let  hint = hint_data.downcast_ref::<Hint>()?;
match  hint {
Hint::Starknet(hint) => {
// Starknet 특화 힌트 (syscall, cheatcode)
match  hint {
StarknetHint::SystemCall { system } => self.execute_syscall(system, vm, exec_sco
StarknetHint::Cheatcode { .. } => self.execute_cheatcode(...),
}
}
Hint::Core(core_hint) => {
// 코어 힌트 (수학, 메모리 연산 등)
execute_core_hint_base(vm, exec_scopes, core_hint, self.no_temporary_segments)
}
Hint::External(hint) => {
// 외부 힌트 (사용자 정의)
self.execute_external_hint(vm, hint)
```

```
}
}
}
}
```

3. Syscall 실행

```
fn execute_syscall(&mut self, system: &ResOperand, vm: &mut VirtualMachine,
exec_scopes: &mut ExecutionScopes) -> Result<(), HintError> {
let system_ptr = extract_relocatable(vm, system)?;
let mut system_buffer = MemBuffer::new(vm, system_ptr);
let selector = system_buffer.next_felt252()?.to_bytes_be();
let mut gas_counter = system_buffer.next_usize()?;
let selector = std::str::from_utf8(&selector).unwrap().trim_start_matches('\0')
match selector {
"StorageRead" => self.storage_read(gas_counter, address, key),
"StorageWrite" => self.storage_write(gas_counter, address, key, value),
"EmitEvent" => self.emit_event(gas_counter, keys, data),
"CallContract" => self.call_contract(gas_counter, contract_address, entry_point,
// ... 기타 syscall들
}
}
```

Starknet 상태 관리

1. StarknetState 구조

```
#[derive(Clone, Default)]
pub struct StarknetState {
/// 컨트랙트별 시뮬레이션된 스토리지 값들
storage: HashMap<Felt252, HashMap<Felt252, Felt252>>,
/// 컨트랙트 주소에서 클래스 해시로의 매핑
deployed_contracts: HashMap<Felt252, Felt252>,
/// 컨트랙트 주소별 로그들
logs: HashMap<Felt252, ContractLogs>,
/// 시뮬레이션된 실행 정보
exec_info: ExecutionInfo,
/// 블록 번호별 클래스 해시 히스토리
block_hash: HashMap<u64, Felt252>,
}
```

2. 실행 컨텍스트 관리

```
impl StarknetState {
/// 컨텍스트의 주소들 교체
pub fn open_caller_context(&mut self, (new_contract_address, new_caller_addre
-> (Felt252, Felt252) {
```

```
let old_contract_address = std::mem::replace(&mut self.exec_info.contract_addr
let old_caller_address = std::mem::replace(&mut self.exec_info.caller_address,
(old_contract_address, old_caller_address)
}
/// 컨텍스트의 주소들 복원
pub fn close_caller_context(&mut self, (old_contract_address, old_caller_addr
self.exec_info.contract_address = old_contract_address;
self.exec_info.caller_address = old_caller_address;
}
}
```

실행 결과 처리

1. RunResult 구조

```
#[derive(Debug, Eq, PartialEq, Clone)]
pub struct RunResult {
pub gas_counter: Option<Felt252>,
pub memory: Vec<Option<Felt252>>,
pub value: RunResultValue,
pub used_resources: ExecutionResources,
pub profiling_info: Option<ProfilingInfo>,
}
```

```rust
#[derive(Debug, Eq, PartialEq, Clone)]
pub enum RunResultValue {
/// 성공적인 실행, 비암시적 반환값들의 메모리
Success(Vec<Felt252>),
/// 패닉으로 인한 실행 실패, 에러 데이터
Panic(Vec<Felt252>),
}
```

2. 반환값 처리

```rust
pub fn handle_main_return_value(
inner_type_size: Option<i16>,
values: Vec<Felt252>,
cells: &[Option<Felt252>],
) -> RunResultValue {
if let Some(inner_type_size) = inner_type_size {
// 패닉 래퍼가 포함된 함수
if values[0] == Felt252::from(0) {
// 성공적인 실행
let inner_ty_size = inner_type_size as usize;
let skip_size = values.len() - inner_ty_size;
RunResultValue::Success(values.into_iter().skip(skip_size).collect())
```

```
} else {
// 패닉 발생
let  err_data_start = values[values.len() - 2].to_usize().unwrap();
let  err_data_end = values[values.len() - 1].to_usize().unwrap();
RunResultValue::Panic(
cells[err_data_start..err_data_end]
.iter()
.cloned()
.map(Option::unwrap)
.collect(),
)
}
} else {
// 패닉 래퍼 없음 - 항상 성공
RunResultValue::Success(values)
}
}
```

성능 프로파일링

1. ProfilingInfo 수집

```
#[derive(Debug, Eq, PartialEq, Clone)]
```

```rust
pub struct ProfilingInfo {
/// 각 Sierra 문에서 발생한 트레이스 스텝 수
pub sierra_statement_weights: UnorderedHashMap<StatementIdx, usize>,
/// 각 스택 트레이스의 가중치 맵
pub stack_trace_weights: OrderedHashMap<Vec<usize>, usize>,
/// 스코프된 Sierra 문 가중치 (호출 스택 정보 포함)
pub scoped_sierra_statement_weights: OrderedHashMap<(Vec<usize>, StatementIdx),
}
```

2. 프로파일링 설정

```rust
#[derive(Debug, Eq, PartialEq, Clone)]
pub struct ProfilingInfoCollectionConfig {
/// 수집할 스택 트레이스의 최대 깊이
pub max_stack_trace_depth: usize,
/// 스코프된 Sierra 문 가중치 수집 여부
pub collect_scoped_sierra_statement_weights: bool,
}
```

3. 트레이스 기반 프로파일링

```rust
impl ProfilingInfo {
pub fn from_trace(
```

```
builder: &RunnableBuilder,

load_offset: usize,

profiling_config: &ProfilingInfoCollectionConfig,

trace: &[RelocatedTraceEntry],

) -> Self {

// 트레이스 엔트리를 순회하며 각 Sierra 문의 실행 횟수 계산

// 함수 호출 스택 추적

// 스코프된 가중치 수집 (옵션)

}

}
```

실행 최적화 기법

1. 메모리 최적화

- **세그먼트 재사용**: 임시 세그먼트 할당 최소화
- **지연 할당**: 필요시에만 메모리 세그먼트 생성
- **버퍼 풀링**: 자주 사용되는 버퍼 재사용

2. 가스 최적화

- **사전 계산**: 함수 호출 전 필요 가스량 계산
- **배치 처리**: 여러 연산을 한 번에 처리하여 오버헤드 감소
- **캐싱**: 가스 비용 계산 결과 캐싱

3. 힌트 최적화

- **힌트 캐싱**: 동일한 힌트의 반복 실행 최적화
- **배치 실행**: 관련된 힌트들을 함께 실행
- **지연 실행**: 필요시까지 힌트 실행 지연

4. 프로파일링 최적화

- **선택적 수집**: 필요한 프로파일링 정보만 수집
- **압축**: 프로파일링 데이터 압축 저장
- **스트리밍**: 대용량 트레이스의 스트리밍 처리

에러 처리 및 디버깅

1. RunnerError 타입

```
#[derive(Debug, Error)]

pub  enum RunnerError {

#[error(transparent)]

BuildError(#[from] BuildError),

#[error("Not enough gas to call function.")]

NotEnoughGasToCall,

#[error("Function param {param_index} only partially contains argument {arg_inde

ArgumentUnaligned { param_index: usize, arg_index: usize },

#[error("Function expects arguments of size {expected} and received {actual} ins

ArgumentsSizeMismatch { expected: usize, actual: usize },
```

```
#[error(transparent)]
CairoRunError(#[from] Box<CairoRunError>),
}
```

2. 패닉 트레이스백

```
pub struct CairoHintProcessor<'a> {
// ...
/// 패닉 트레이스 힌트 호출로 설정된 트레이스백
pub panic_traceback: Vec<(Relocatable, Relocatable)>,
}
```

3. 실행 리소스 추적

```
#[derive(Debug, Eq, PartialEq, Clone, Default)]
pub struct StarknetExecutionResources {
/// 기본 실행 리소스
pub basic_resources: ExecutionResources,
/// 사용된 syscall들
pub syscalls: HashMap<String, usize>,
}
```

결론

Cairo 실행 환경은 복잡한 메모리 관리, 정교한 가스 계산, 그리고 다양한 힌트 처리를 통해 Cairo 프로그램의 안전하고 효율적인 실행을 보장합니다. VM 인터페이스는 추상화를 통해 다양한 실행 컨텍스트(일반 Cairo, Starknet)를 지원하며, 프로파일링과 최적화 기능을 통해 성능 분석과 개선을 가능하게 합니다.